МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТк

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ИИ26(з)

Выполнил
Т.В.
Данилюк,
студент группы ИИ26

Проверила
К.В. Андренко,
ст. преп. кафедры ИИТ,
«___k_____2026 г.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Код программы:

Содержимое в файле main.py

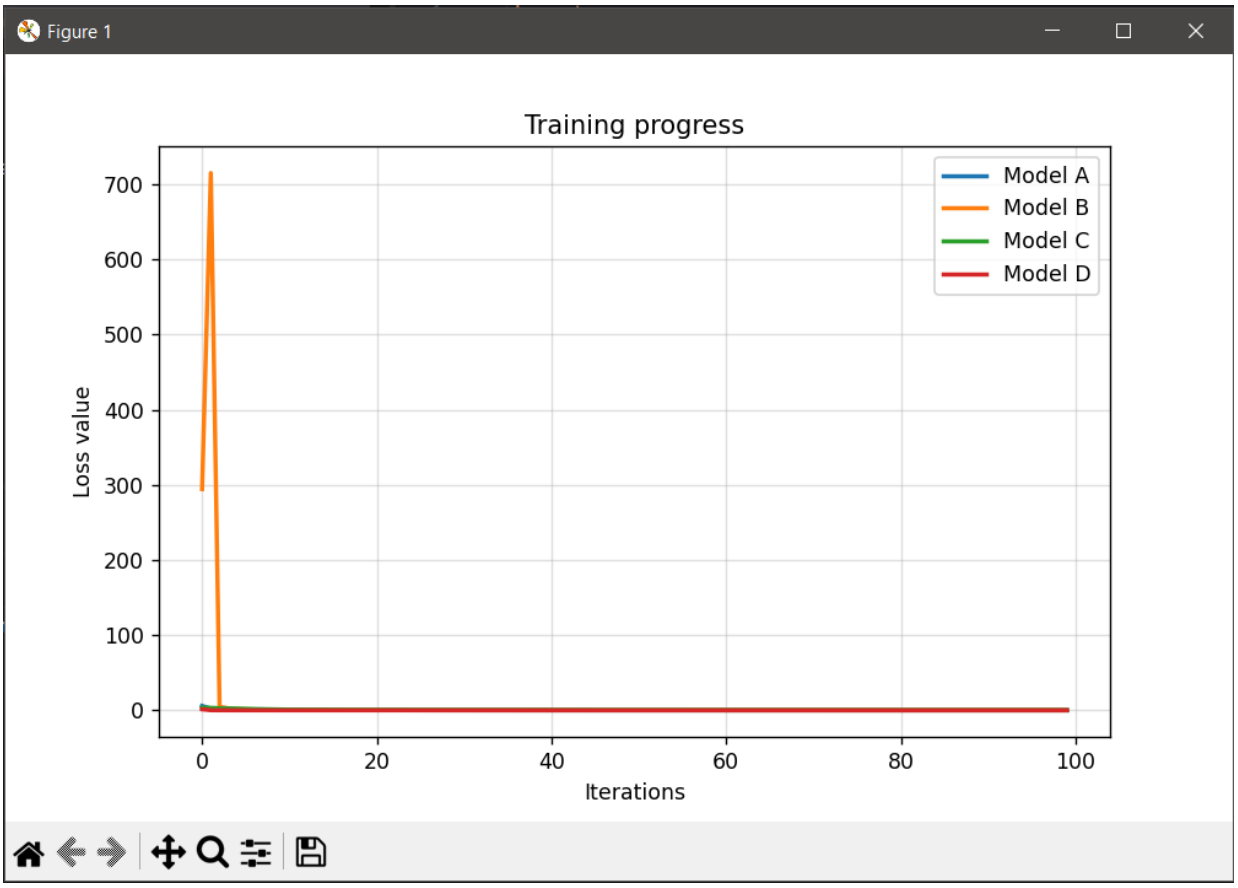
```
import numpy as np
import matplotlib.pyplot as plt

points = np.array([
    [1, 4],
    [-1, 4],
    [1, -4],
    [-1, -4]
], dtype=float)

labels = np.array([0, 0, 0, 1], dtype=float)

max_steps = 100
limit = 0.01


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


def loss(prob, target):
    eps = 1e-9
    return -(target * np.log(prob + eps) + (1 - target) * np.log(1 - prob + eps))


def linear_fixed(data, y, rate):
    weights = np.random.randn(2)
    bias = np.random.randn()
    history = []

    for epoch in range(max_steps):
        err_sum = 0
        for vector, target in zip(data, y):
            z = np.dot(weights, vector) - bias
            pred = z
            delta = target - pred
            err_sum += delta ** 2

            weights += rate * delta * vector
            bias -= rate * delta

        history.append(err_sum / len(data))
        if history[-1] <= limit:
            break
```

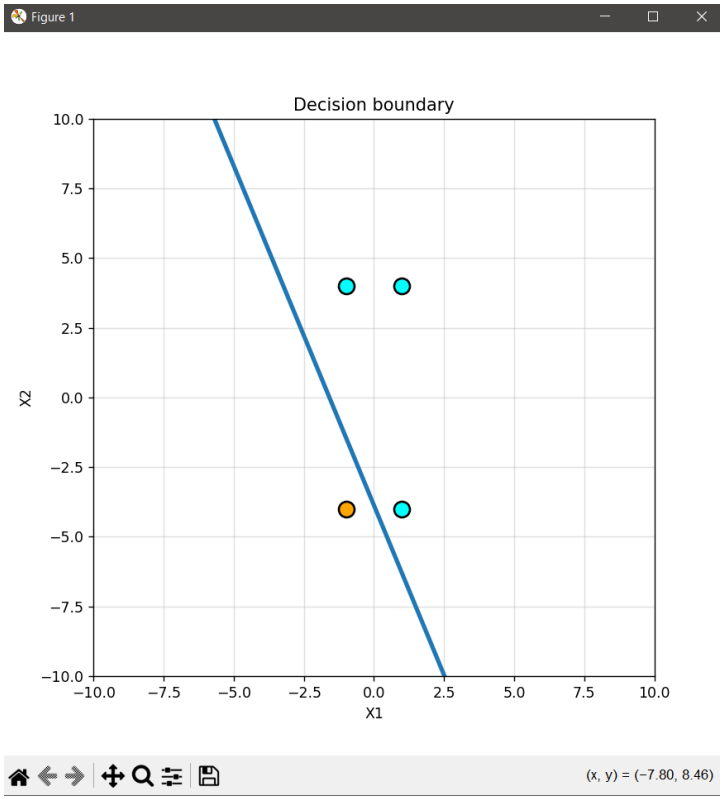
```
    return weights, bias, history


def linear_dynamic(data, y):
    weights = np.random.randn(2)
    bias = np.random.randn()
    history = []
    step = 1

    for epoch in range(max_steps):
        err_sum = 0
        for vector, target in zip(data, y):
            rate = 0.5 / step
            step += 1

            z = np.dot(weights, vector) - bias
            pred = z
            delta = target - pred
            err_sum += delta ** 2

            weights += rate * delta * vector
            bias -= rate * delta

        history.append(err_sum / len(data))
        if history[-1] <= limit:
            break

    return weights, bias, history


def logistic_fixed(data, y, rate):
    weights = np.random.randn(2)
    bias = np.random.randn()
    history = []

    for epoch in range(max_steps):
        err_sum = 0
        for vector, target in zip(data, y):

            z = np.dot(weights, vector) - bias
            prob = sigmoid(z)

            err_sum += loss(prob, target)

            grad = (prob - target)
            weights -= rate * grad * vector
            bias += rate * grad

        history.append(err_sum / len(data))
        if history[-1] <= limit:
            break

    return weights, bias, history


def logistic_dynamic(data, y):
    weights = np.random.randn(2)
```

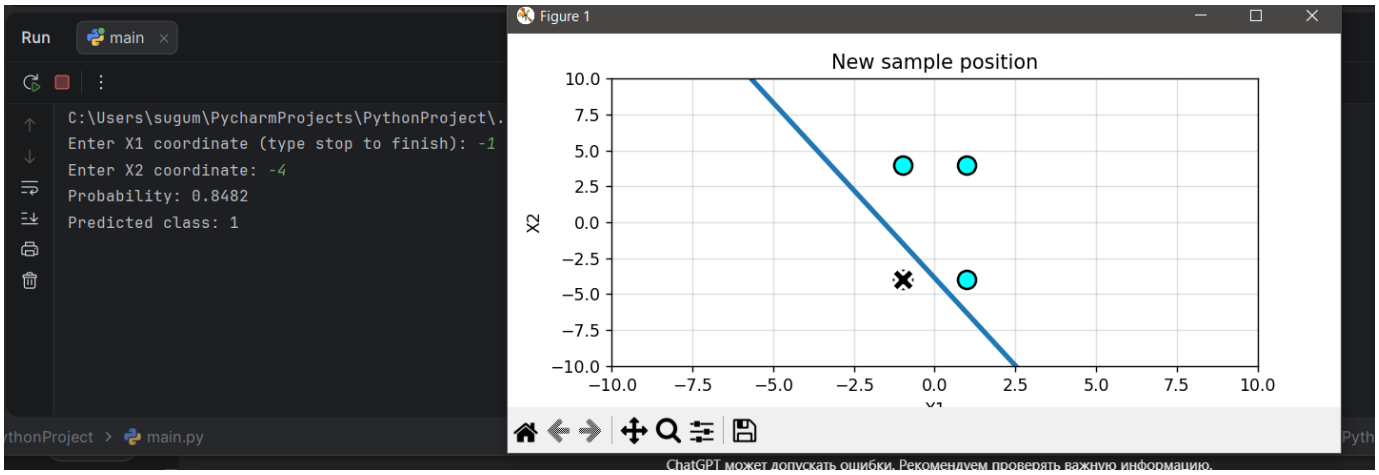
```
    bias = np.random.randn()
    history = []
    step = 1

    for epoch in range(max_steps):
        err_sum = 0
        for vector, target in zip(data, y):

            rate = 1 / step
            step += 1

            z = np.dot(weights, vector) - bias
            prob = sigmoid(z)

            err_sum += loss(prob, target)

            grad = (prob - target)
            weights -= rate * grad * vector
            bias += rate * grad

        history.append(err_sum / len(data))
        if history[-1] <= limit:
            break

    return weights, bias, history


w1, b1, err1 = linear_fixed(points, labels, 0.01)
w2, b2, err2 = linear_dynamic(points, labels)
w3, b3, err3 = logistic_fixed(points, labels, 0.01)
w4, b4, err4 = logistic_dynamic(points, labels)

plt.figure(figsize=(8,5))
plt.plot(err1, linewidth=2, label="Model A")
plt.plot(err2, linewidth=2, label="Model B")
plt.plot(err3, linewidth=2, label="Model C")
plt.plot(err4, linewidth=2, label="Model D")
plt.xlabel("Iterations")
plt.ylabel("Loss value")
plt.title("Training progress")
plt.legend()
plt.grid(True, alpha=0.4)
plt.show()

plt.figure(figsize=(7,7))

for i in range(len(points)):
    color = "orange" if labels[i] == 1 else "cyan"
    plt.scatter(points[i,0], points[i,1], color=color, s=120, edgecolors='black', linewidth=1.5)

line_x = np.linspace(-10, 10, 300)

if w4[1] != 0:
    line_y = (b4 - w4[0]*line_x) / w4[1]
    plt.plot(line_x, line_y, linewidth=3)

plt.xlim(-10, 10)
plt.ylim(-10, 10)
```

```
plt.grid(True, alpha=0.4)
plt.title("Decision boundary")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()


def predict_point(x1, x2):

    value = w4[0]*x1 + w4[1]*x2 - b4
    probability = sigmoid(value)
    predicted_class = 1 if probability >= 0.5 else 0

    print(f"Probability: {probability:.4f}")
    print(f"Predicted class: {predicted_class}")

    plt.figure(figsize=(7,7))

    for i in range(len(points)):
        color = "orange" if labels[i] == 1 else "cyan"
        plt.scatter(points[i,0], points[i,1], color=color, s=120, edgecolors='black', linewidth=1.5)

    if w4[1] != 0:
        plt.plot(line_x, line_y, linewidth=3)

    plt.scatter(x1, x2, color="black", s=200, marker="X", edgecolors='white', linewidth=2,
zorder=5)

    plt.xlim(-10, 10)
    plt.ylim(-10, 10)
    plt.grid(True, alpha=0.4)
    plt.title("New sample position")
    plt.xlabel("X1")
    plt.ylabel("X2")
    plt.show()


while True:
    val1 = input("Enter X1 coordinate (type stop to finish): ")

    if val1.lower() == "stop":
        break

    val2 = input("Enter X2 coordinate: ")

    try:
        predict_point(float(val1), float(val2))
    except:
        print("Input format error")
```

Результата работы программы:







Вывод: В ходе работы были получены практические навыки реализации алгоритмов машинного обучения, анализа процесса обучения моделей и визуализации результатов классификации.